A Beginner's Walkthrough

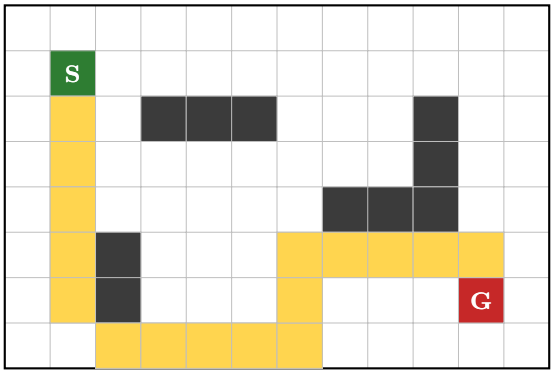
of the *JPS4 Snake-Game Benchmark* Project

Dijkstra, A*, and Jump Point Search for 4-Connected Grids

Companion guide for the ADA course project

Jazib Waqas · Salman Adnan · Hunain Abbas

April 11, 2026



From start (S) to goal (G), avoiding walls. That's the whole problem — and it's the core of three famous algorithms.

Contents

Preface **3**

1 What problem are we solving? **4**
1.1 The plain-English version . . . 4
1.2 Why anyone cares . . . 4
1.3 What makes a grid a "grid graph" . . . 4
1.4 The two things we measure . . . 5

2 Building blocks shared by all three algorithms **6**
2.1 The "frontier" idea . . . 6
2.2 The priority queue . . . 6
2.3 Cost so far: g . . . 6
2.4 Estimated cost to go: h . . . 6
2.5 Total priority: $f = g + h$ . . . 7
2.6 The three knobs . . . 7

3 Dijkstra's algorithm **8**
3.1 The idea in one sentence . . . 8
3.2 A worked example . . . 8
3.3 Pseudocode . . . 8
3.4 What's good, what's not . . . 8

4 A* search **9**
4.1 The idea in one sentence . . . 9
4.2 Dijkstra vs. A*, same grid, side by side . . . 9
4.3 Why does adding h still give the shortest path? . . . 9
4.4 Pseudocode . . . 9

5 Jump Point Search for 4-connected grids (JPS4) **11**
5.1 The path-symmetry problem . . . 11
5.2 Canonical ordering: "there is only one path" . . . 11
5.3 Natural neighbours and forced neighbours . . . 11
5.4 Jumping . . . 12
5.5 A visual intuition of jumping . . . 13
5.6 What JPS4 is good at and what it is not . . . 13

6 A short summary of the research paper **14**
6.1 The gap it fills . . . 14
6.2 What the paper contributes . . . 14
6.3 What the paper measures . . . 14

7 Project architecture: the code tour **15**
7.1 The layered view . . . 15
7.2 `helper.py` — the primitives . . . 15
7.3 `pathing_grid.py` — the three algorithms . . . 16
7.4 `snake_game_jps4.py` — the visual front-end . . . 17
7.5 `benchmark_interim.py` and `run_preliminary_benchmark.py` . . . 17

8 How a search call flows through the code **18**
8.1 Step-by-step in words . . . 18

9 The Snake game, in detail **20**
9.1 The game grid . . . 20
9.2 The tick cycle . . . 20
9.3 The compare mode . . . 20
9.4 The two boards . . . 20
9.5 Why the snake game is a good demo for JPS4 . . . 21

10 The benchmarking methodology **22**
10.1 Fair comparison . . . 22
10.2 What each script does . . . 22
10.3 Start and goal placement . . . 22
10.4 What is reported . . . 22
10.5 What the interim numbers showed . . . 22

11 Glossary **24**

Glossary **24**

Preface: How to read this document

This guide assumes **zero prior knowledge**. You do not need to know what a graph is, what an algorithm is, or what Python looks like. Every new term is introduced the first time it appears, with a small picture whenever possible.

The document is organised like a staircase. Each section builds on the one before:

1. First, we define the problem in plain English (§1).
2. Then, we introduce the ideas (graph, priority queue, heuristic) that all three algorithms share (§2).
3. Next, we walk through each algorithm in turn — Dijkstra, A*, and JPS4 — with pictures showing exactly which cells the algorithm touches and why (§3–§5).
4. Then, we summarise the research paper the project is based on (§6).
5. Finally, we open the project itself: every file, every class, every function, and how they connect (§7–§10).

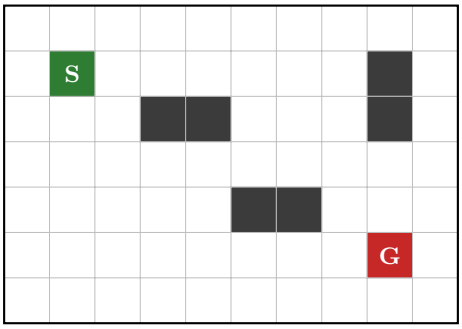
How to use this in one sentence

If any page is confusing, go back one section — this document is designed so that nothing is used before it has been explained.

1. What problem are we solving?

1.1 The plain-English version

Imagine a chessboard with some squares blocked by walls. You are standing on one square (the **start**, marked **S**). You want to reach another square (the **goal**, marked **G**). You can only move one step at a time, and only in four directions: up, down, left, right. You cannot step diagonally and you cannot walk through walls.



The question is: **what is the shortest route from S to G?**

That's it. That is the entire problem. It sounds easy because a human can glance at a small grid and see a good route almost instantly. Computers cannot. A computer has to check squares one at a time, and on a big grid it has to check a *lot* of them — possibly tens of thousands. How it decides *which squares to check first* is what makes an algorithm fast or slow.

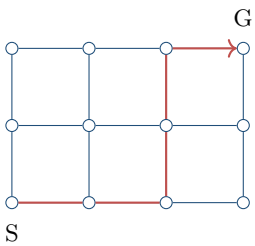
1.2 Why anyone cares

This problem sounds like a puzzle, but it appears everywhere:

- **Video games.** In almost every game with characters that walk around, there is a pathfinder running behind the scenes. When a monster chases you in a dungeon, something like this is deciding where it steps next.
- **Robotics.** A delivery robot inside a warehouse needs to get from shelf A to shelf B without bumping into shelves or other robots.
- **Navigation apps.** Google Maps is a fancier cousin of this problem, on a road network instead of a grid.
- **Logistics.** Routing trucks, planning forklift movements, scheduling crane moves in a shipping yard.

1.3 What makes a grid a "grid graph"

When a computer scientist looks at our board, they don't see squares — they see a *graph*. A graph is just a collection of **nodes** (dots) connected by **edges** (lines). Each free square becomes one dot, and we draw a line between two dots whenever you can walk directly from one square to the other (so: adjacent, free, not separated by a wall).



Every step is an edge. Every edge has the same *cost*: 1. So the "shortest path" is the path with the fewest steps. Because every step costs the same, we call this a **uniform-cost 4-connected grid graph**. Memorise that phrase; it's the exact setting all three algorithms are built for.

1.4 The two things we measure

Whenever we compare algorithms in this project, we measure two numbers:

The two metrics

(1) Node expansions — how many grid cells the algorithm "looked at" (pulled out of its to-do list to process). This is a pure count, independent of the machine.

(2) Search time — actual wall-clock time in milliseconds. This depends on the machine and the programming language.

Expansions are the cleaner number because they reflect the *algorithm*, not the computer. Search time is what the user eventually experiences, but it's noisy.

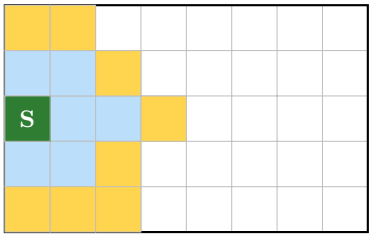
2. Building blocks shared by all three algorithms

All three algorithms we study — Dijkstra, A*, and JPS4 — share the same skeleton. They differ only in three knobs, which we'll identify at the end of this section. First let's build the skeleton.

2.1 The "frontier" idea

Imagine standing at S and gradually exploring outward. At any moment, there are some squares you have already visited, and some squares that are on the *border* between the explored region and the unknown. That border is called the **frontier** (or "open list").

At every step the algorithm does one thing: *pick one square from the frontier, mark it visited, and add its free neighbours to the frontier.* Repeat until the goal pops out.



blue = visited
yellow = frontier

The cleverness is in *which* frontier cell you pick next. That one decision is the whole story.

2.2 The priority queue

The frontier is not a plain list. It is a **priority queue**: a container where every item has a number attached (a "priority"), and whenever you ask for an item you always get the one with the smallest number.

A good mental picture: a pile of sticky notes, each with a number. You always pick the one with the smallest number off the top of the pile. Inserting a new note is cheap, and so is removing the smallest.

Why a priority queue?

Different algorithms will attach different priority numbers to each frontier cell. The priority queue doesn't care what the number *means* — it just gives you the smallest one. That is how the *same skeleton* can behave like three different algorithms, simply by choosing a different priority formula.

In the project's code (`helper.py`), the priority queue is Python's built-in `heapq`, and each frontier entry is a `SearchNode` object holding the priority, the running cost, the cell, and a back-pointer to the parent cell.

2.3 Cost so far: g

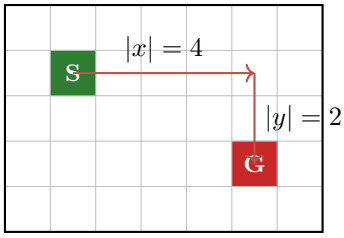
For every cell we visit, we record **how many steps it took to get there from S**. That number is called g (for "cost so far"). The start has $g = 0$; its neighbours have $g = 1$; their neighbours have $g = 2$; and so on.

2.4 Estimated cost to go: h

Some algorithms also want to *guess* how far the goal still is from a given cell. That guess is called h (for "heuristic"). In a 4-connected grid, a very good guess is the **Manhattan distance**:

$$h(\text{cell}) = |\text{cell}.x - \text{goal}.x| + |\text{cell}.y - \text{goal}.y|$$

That is: pretend there are no walls and count the minimum number of cardinal steps you would need. Because you can't do better than that (walls only make things harder), Manhattan distance is *never an overestimate* — a property called **admissibility**. Admissibility is the magic ingredient that lets A* still guarantee the shortest path even though it is guessing.



Manhattan $= 4 + 2 = 6$

2.5 Total priority: $f = g + h$

The priority the algorithms use is a number called f, which is just $g + h$:

$$f(\text{cell}) = g(\text{cell}) + h(\text{cell})$$

That is: "how many steps we already spent to reach this cell, plus our best guess of how many more we'll need". When the priority queue gives us the cell with the smallest f, it's giving us the cell that *could* lie on the shortest path, because no other candidate currently looks cheaper.

2.6 The three knobs

Now we can describe the three algorithms in a single sentence each, by saying how they set those knobs:

Algorithm	h (heuristic)	**Successors (neighbours)**	**Priority**
Dijkstra	always 0	all 4 cardinal neighbours	$f = g$
A*	Manhattan	all 4 cardinal neighbours	$f = g + h$
JPS4	Manhattan	only *jump points* (see §5)	$f = g + h$

Three lines. That is literally the difference. In the code (`pathing_grid.py`), the same function `astar_jps` runs all three; it receives a `heuristic` function and a `successors` function as arguments, and the caller swaps them depending on which algorithm the user asked for.

3. Dijkstra's algorithm

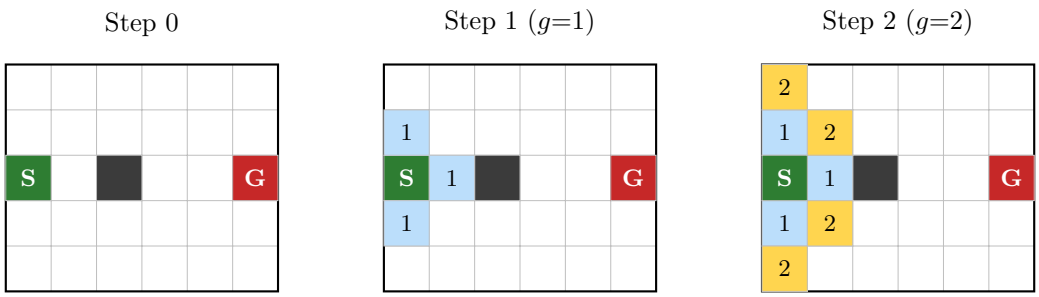
3.1 The idea in one sentence

Spread out from the start in a perfectly circular ripple, one step at a time, until you hit the goal.

Dijkstra does not use a heuristic. It has no idea where the goal is. It just processes cells in order of increasing distance from the start, like a water ripple. Because every cell at distance d is processed before any cell at distance $d + 1$, the very first time we pull the goal out of the priority queue, we know we've found the shortest path.

3.2 A worked example

Start at S. Its four neighbours all go into the frontier with $g = 1$. We pick one (any one), process it, adding its unvisited neighbours with $g = 2$. And so on.



After enough ripples, the goal is reached. Following the back-pointers from G back to S gives the shortest route.

3.3 Pseudocode

```
def dijkstra(start, goal, grid):
    frontier = PriorityQueue()
    frontier.push((g=0, start))
    came_from = {start: None}
    best_g    = {start: 0}

    while not frontier.empty():
        g, cell = frontier.pop()      # smallest g first
        if cell == goal:
            return reconstruct(came_from, goal)
        for nb in cell.neighbours():         # four cardinal
            if nb is wall: continue
            new_g = g + 1
            if new_g < best_g.get(nb, inf):
                best_g[nb]   = new_g
                came_from[nb] = cell
                frontier.push((new_g, nb))
    return None    # unreachable
```

3.4 What's good, what's not

+ **Always correct.** It will find the shortest path every time, as long as one exists.
+ **Conceptually simple.** Nothing fancy; no heuristic to tune.
− **Wasteful.** It explores in *every* direction, including directions pointing away from the goal. On a big map it may visit tens of thousands of cells before stumbling into G. This is the exact weakness A* was invented to fix.

4. A* search

4.1 The idea in one sentence

Dijkstra, but bias the ripple so it grows faster in the direction of the goal.

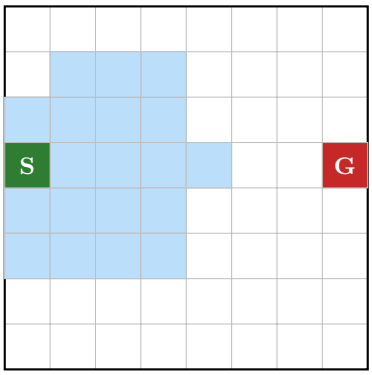
A* uses the Manhattan-distance heuristic h to break ties. Instead of ordering the frontier by g alone ("distance from start"), it orders by $f = g + h$ ("distance from start plus estimated distance to goal"). Cells that point toward G get a lower priority number and come out of the queue sooner. Cells that point away get pushed to the back.

The result: instead of a round ripple, A* expands in an elongated shape that stretches toward the goal.

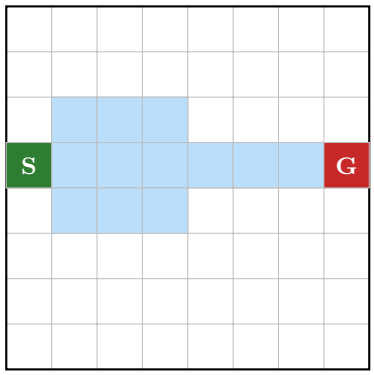
4.2 Dijkstra vs. A*, same grid, side by side

Each coloured cell is one that the algorithm actually pulled out of the frontier ("expanded"). The darker the colour, the earlier it was expanded.

Dijkstra (round ripple)



A* (biased toward goal)



On an open map the difference is dramatic. Dijkstra expands dozens of cells it didn't need to; A* takes an almost-straight line.

4.3 Why does adding h still give the shortest path?

This is the subtlety that made A* famous in 1968 (Hart, Nilsson, and Raphael). The argument is: as long as h never *over-estimates* the true remaining cost, the first time A* pops the goal out of the priority queue, the g value attached to it is already optimal. Manhattan distance on a 4-connected grid is never an over-estimate (walls only make things harder), so we're safe.

The rule for a safe heuristic

$h(\text{cell}) \leq$ actual shortest distance from cell to goal, *for every cell.*
If that holds, A* is guaranteed to find the shortest path. This property is called **admissibility**.

4.4 Pseudocode

```
def astar(start, goal, grid):
    frontier = PriorityQueue()
    frontier.push((f=h(start), g=0, start))
    came_from, best_g = {start: None}, {start: 0}

    while not frontier.empty():
        f, g, cell = frontier.pop()
```

```
        if cell == goal:
            return reconstruct(came_from, goal)
        for nb in cell.neighbours():
            if nb is wall: continue
            new_g = g + 1
            if new_g < best_g.get(nb, inf):
                best_g[nb]   = new_g
                came_from[nb] = cell
                frontier.push((new_g + h(nb), new_g, nb))
    return None
```

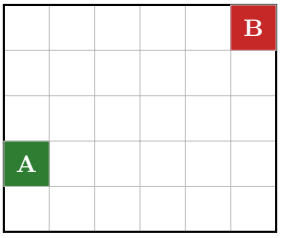
Notice that the only difference from Dijkstra is line by line identical *except* the priority pushed into the queue: `new_g + h(nb)` instead of just `new_g`. That is the whole change.

5. Jump Point Search for 4-connected grids (JPS4)

This is the most interesting algorithm in the project, and also the trickiest. Pay close attention to the setup — the algorithm itself is short, but it only makes sense after you understand the **path-symmetry problem** it is designed to solve.

5.1 The path-symmetry problem

Look at a small open section of grid:

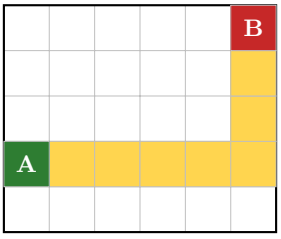


How many shortest paths are there from A to B? The answer: *very many*. Any sequence with 5 rights and 3 ups, in any order, is a valid shortest path. All of them have the same length (8). They are all *equivalent* — none is better than any other. A* will expand a lot of cells to compare these equivalent paths, even though it doesn't matter which one wins.

This duplication is called **path symmetry**. It is pure wasted work: if the grid is open, there is nothing to decide between the symmetric alternatives, so A* shouldn't even be looking at them.

5.2 Canonical ordering: "there is only one path"

The fix is to pretend, artificially, that only one ordering is allowed. JPS4 picks a rule called **horizontal-first**: "do all your horizontal steps before any vertical steps". Under this rule, on an open map, there is exactly *one* shortest path from A to B, not many:



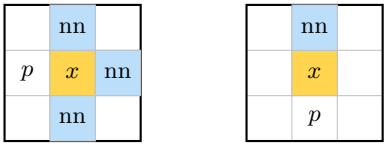
All the other shortest paths still exist in principle, but the algorithm refuses to consider them. This does not hurt optimality — they all have the same length anyway.

5.3 Natural neighbours and forced neighbours

If every cell still produced four neighbours, the canonical ordering alone wouldn't help. JPS4 also **prunes** the neighbour list according to how you arrived at the current cell. The rules come straight from Baum (2025), Algorithm 2:

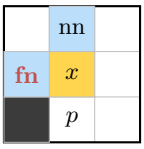
You arrived moving...	**Natural neighbours**
Horizontally (left or right)	All three non-parent neighbours (forward, up, down)
Vertically (up or down)	Only the one cell straight ahead in the same direction

Arriving **horizontally** Arriving **vertically**



(Here p is the parent cell we arrived from, x is the current cell, and "nn" marks a natural neighbour.)

Now here's the catch: pruning so aggressively *can* miss paths if there are obstacles. Look at this situation:



We arrived at x moving up. Normally we'd only keep the cell straight above x (the "nn"). But the cell to the left of x (labelled **fn**, for *forced neighbour*) has become interesting because the cell to the left of p is a wall. If we don't expand left at x, there's no other way to get around that wall. So we add it back in as a *forced* neighbour — a neighbour that would normally be pruned but is needed because an obstacle broke the symmetry.

The rule for forced neighbours (vertical movement)

Moving vertically, check each of the two sideways neighbours of x (left and right). If the corresponding sideways neighbour of p is a *wall*, then the one at x becomes **forced**. (Horizontal movement never produces forced neighbours.)

5.4 Jumping

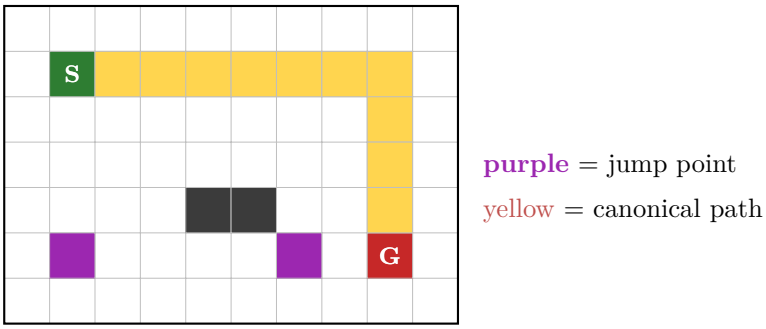
The final trick: instead of walking one step at a time, JPS4 *skips ahead*. When moving vertically, it keeps stepping until one of three things happens:

1. It hits a wall or the edge of the grid → dead end, stop.
2. It reaches the goal → success, stop.
3. It finds a cell that has a *forced* neighbour → stop here; this cell is a **jump point** and must be added to the frontier.

All the cells in between are silently skipped. They never enter the priority queue and never add to the expansion count. That is where the speed comes from.

Horizontal moves are handled slightly differently — they advance just one step and always return the result. This asymmetry is the reason the algorithm is called **horizontal-first** JPS (HJPS) in Baum's paper.

5.5 A visual intuition of jumping



Without jumping, A* expands every yellow cell one at a time — ten expansions. JPS4 expands only the start, the turning points, and the goal — maybe three or four expansions. The yellow cells still appear in the final path (JPS4 fills them in at the end), but they were never in the priority queue.

5.6 What JPS4 is good at and what it is not

+ **Long open corridors.** This is where jumping shines: one jump can replace dozens of A* expansions.
+ **Still optimal.** Baum proves (Theorem 4.2 in the paper) that JPS4 always returns a shortest path.
− **Dense noise.** When obstacles are sprinkled everywhere (30%+), corridors are short, so each "jump" barely advances, and the extra work of checking for forced neighbours at every step starts to hurt.
− **Completely empty maps.** Surprisingly, A* can beat JPS4 on obstacle-free maps because the heuristic alone is enough to walk almost straight to the goal, and JPS4's per-step overhead is pure loss. This is the one result Baum warns about explicitly.

6. A short summary of the research paper

Title. Jump Point Search Pathfinding in 4-connected Grids.
Author. Johannes Baum (Google), January 2025.
Where. arXiv preprint 2501.14816.

6.1 The gap it fills

The original Jump Point Search (JPS) was invented by Harabor and Grastien in 2011 for **8-connected** grids, where characters can move diagonally. But many video games and some robotics settings only allow the four cardinal directions (up, down, left, right) — a **4-connected** grid. The original JPS does not work directly on 4-connected grids, because its pruning rules rely on diagonal moves to "hop over" corners of obstacles. Baum adapts the algorithm by replacing the 8-direction canonical ordering with a **horizontal-first** ordering and rewriting the natural- and forced-neighbour rules from scratch.

6.2 What the paper contributes

1. A formal definition of the horizontal-first canonical ordering on 4-connected grids.
2. New pruning rules (Algorithms 2 and 3 in the paper) that define natural and forced neighbours for only the four cardinal directions.
3. A jump function (Algorithm 5) that returns immediately after one step for horizontal moves and recurses for vertical moves.
4. A proof (Theorem 4.2) that JPS4 is optimal: it always returns a shortest path.
5. Benchmarks on three real-world map sets from the Sturtevant pathfinding benchmark collection — *Dragon Age: Origins*, *Baldur's Gate 2*, and *Rooms* — plus an empty-map set.

6.3 What the paper measures

- On **Baldur's Gate 2**, JPS4 is roughly **10–20**× faster than A* for typical path lengths.
- On **Dragon Age: Origins**, about **10**× faster.
- On **Rooms**, the speedup grows almost linearly with path length, reaching **50**× or more.
- On an **empty map**, A* *beats* JPS4. The empty-map speedup is less than 1.0 (around 0.03–0.14), because JPS4 visits every cell while A* walks almost straight to the goal.

Baum's conclusion is clear: JPS4 is a good drop-in for A* when your environment has meaningful obstacles, but on truly open terrain A* is still the right choice.

One takeaway from the paper

JPS4's speed comes from skipping symmetric paths through open corridors. The denser the "useful" structure of the map (rooms, walls, corridors), the more JPS4 wins. The sparser the structure, the less JPS4 wins — and below a density threshold, A* wins.

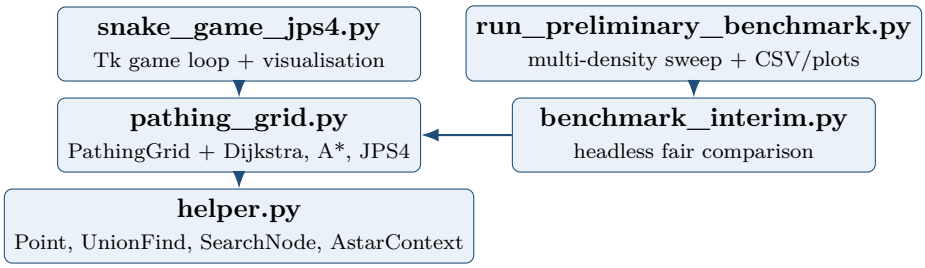
7. Project architecture: the code tour

Now that you understand the three algorithms, let's open the project and walk through every file. The top-level folder looks like this:

```
Proj/
├ Algos_Project_Proposal.pdf
├ README.md
├ requirements.txt
├ benchmark_interim.py
├ run_preliminary_benchmark.py
├ generate_report_figures.py
├ results/
├ report/
└ src/
    └ Snake Game Code/
        ├ helper.py
        ├ pathing_grid.py
        ├ snake_game_jps4.py
        └ Assets/
```

The heart of the project is the three files inside `src/Snake Game Code/`. Everything else — benchmarks, figure scripts, reports — exists to drive or document those three files.

7.1 The layered view



Arrows read "uses". Higher boxes depend on lower boxes.

7.2 `helper.py` — the primitives

This file defines the low-level pieces every search uses. Four things live here:

`Point`. A tiny class that holds an (x, y) pair. It supports addition (`p1 + p2`), equality, hashing (so points can be used as dictionary keys), the four-cardinal neighbour list (`neumann_neighborhood`), and the Manhattan distance. It has one subtle method, `direction_to(other)`, which returns a unit-step direction (dx, dy) — used by JPS4 to figure out which way it is moving.

`UnionFind`. A classic data structure that answers the question "are these two things in the same group?" in almost constant time. The project uses it to pre-check whether the goal is even reachable from the start before running a search. If they are in different connected components

of free cells, there is no point running Dijkstra/A*/JPS4 at all — we already know the answer is "no path".

`SearchNode`. A lightweight record placed onto the priority queue. Holds four things:

- `estimated_cost` — this is $f = g + h$. Python's heap orders on this first.
- `cost` — this is g, used to break ties.
- `node` — the actual grid cell.
- `parent` — the cell we came from (for reconstructing the path at the end).

`AstarContext`. The generic best-first search loop, shared by all three algorithms. Its core is the method `astar_jps(start, successors, heuristic, success)`. Read that signature carefully:

- `start` — where to begin.
- `successors(parent, current)` — a *function* that, given the current cell and its parent, returns the list of `(next_cell, move_cost)` tuples to try next. *This is the knob that distinguishes Dijkstra/A*/JPS4.*
- `heuristic(cell)` — a *function* returning h. Dijkstra passes a function that always returns 0; A* and JPS4 pass a Manhattan-distance function.
- `success(cell)` — a *function* returning `True` if this cell is the goal.

By passing different functions for `successors` and `heuristic`, the same loop becomes Dijkstra, A*, or JPS4. The body of the loop is fewer than 20 lines.

7.3 `pathing_grid.py` — the three algorithms

This is the largest file in the project. It defines a `PathingGrid` class that holds the 2D boolean grid (`True` = blocked) and all three search implementations as methods. The structure:

Method	What it does
`in_bounds, get, set`	Grid bookkeeping.
`can_move_to(p)`	Is cell `p` inside and free?
`heuristic(p1, p2)`	Manhattan distance times the uniform cost.
`neighborhood_points(p)`	The 4 cardinal neighbours of `p`.
`standard_astar_successors`	Returns all 4 free neighbours with cost 1. Used by Dijkstra and A*.
`jps_prune_neighbors`	Implements Baum's Algorithm 2 — natural and forced neighbours.
`jps_jump`	Implements Baum's Algorithm 5 — horizontal returns after 1 step, vertical recurses until forced/goal/wall.
`jps_successors`	Uses the previous two to produce the JPS4 successor list.
`reachable(a, b)`	Union-Find pre-check: are `a` and `b` in the same component?
`get_waypoints_single_goal(s, g, mode)`	**The entry point.** Selects the right successor and heuristic functions based on `mode`, runs `astar_jps`, measures time, records expansions, returns the path.
`get_path_single_goal(s, g, mode)`	Thin wrapper that also validates the path.

7.4 `snake_game_jps4.py` — the visual front-end

This is a full Tkinter-based snake game. The snake's head is the *start*, the food is the *goal*, and the snake's body segments act like walls. Every time the game needs to decide the next move, it calls `PathingGrid.get_path_single_goal` on the current board. The game also has a "compare" button that runs all three algorithms on the current state and draws each algorithm's chosen route in its own colour, so you can visually compare the three side by side:

Algorithm	Colour in the game	Source
Dijkstra	red-pink `#d48888`	`COMPARE_COLORS[0]`
A*	green `#88c488`	`COMPARE_COLORS[1]`
JPS4	blue `#6a9fd4`	`COMPARE_COLORS[2]`

There are two built-in board layouts: an **easy** board with a few scattered walls (20×20) and a **hard** board with a dense maze-like pattern. The hard board is where JPS4's advantage over A* is most visible.

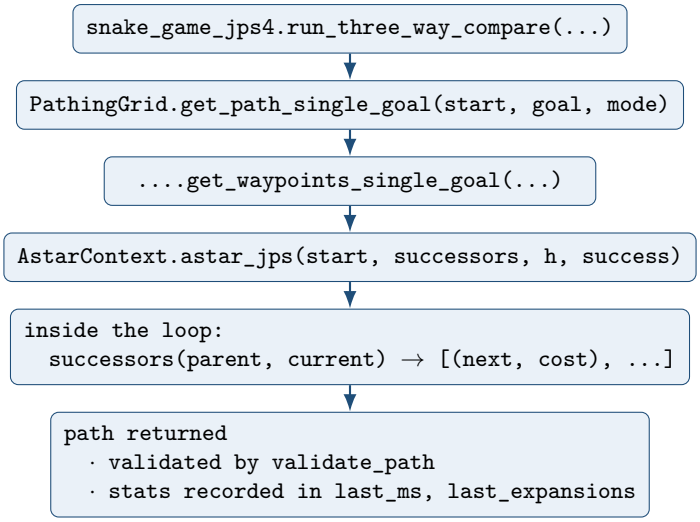
7.5 `benchmark_interim.py` and `run_preliminary_benchmark.py`

These two scripts drive `PathingGrid` headlessly (no GUI) and record CSVs of node expansions and wall-clock time for each algorithm.

- `benchmark_interim.py` runs a single density setting on a single grid size. It generates a random grid, picks a start and goal far apart, verifies they're connected, and runs all three algorithms on the *same* instance so the comparison is fair.
- `run_preliminary_benchmark.py` imports the helpers from the interim script and runs multiple densities in a single pass, dumping CSV, JSON, a LaTeX table snippet, and matplotlib plots into `results/`.

8. How a search call flows through the code

Let's trace what happens when the snake game asks for a path from head to food. This is the single most important flow in the project. Every number in the benchmark comes out of this sequence.



8.1 Step-by-step in words

1. The game has the current head position, the current food position, and a 2D array of the board. It wants the shortest route from head to food that avoids the snake's own body.

2. The game builds a fresh `PathingGrid` for this moment in time, marks the body segments as blocked, and calls `get_path_single_goal(head, food, mode)`. The `mode` string is one of `"dijkstra"`, `"astar"`, or `"jps4"`.

3. `get_path_single_goal` resets the measurement counters (`last_ms` and `last_expansions`) and calls `get_waypoints_single_goal`, which does three cheap checks first:

   - If start and goal are adjacent, return `[start, goal]` immediately.
   - If there's a straight unobstructed line between them, return that line.
   - Otherwise, run the union-find `reachable` check. If the two cells are in different connected components, return `None` (no path).

4. If all the cheap checks fail, we run the search. Here is the key moment where the algorithm is chosen, and it is essentially three lines:

```
if mode == "jps4":
    result = context.astar_jps(start,
        lambda p, n: self.jps_successors(p, n, goal_test),
        heuristic, goal_test)
elif mode == "dijkstra":
    result = context.astar_jps(start,
        lambda p, n: self.standard_astar_successors(n, goal_test),
        lambda _: 0,  # heuristic = 0
        goal_test)
else:   # "astar"
    result = context.astar_jps(start,
        lambda p, n: self.standard_astar_successors(n, goal_test),
        heuristic, goal_test)
```

5. `astar_jps` runs the generic best-first search loop: it pops the lowest-f node, checks if it's the goal, otherwise asks the `successors` function for children, relaxes their costs, and pushes them onto the priority queue. It increments `expansions` by 1 for each pop.
6. When the goal pops out of the queue, `astar_jps` walks back through the `parents` dictionary to reconstruct the path, and returns it.
7. `get_waypoints_single_goal` records the wall-clock time (`last_ms`) and the expansion count (`last_expansions`), then runs one final validation pass — checking that every pair of adjacent points in the path is actually adjacent on the grid and not blocked.
8. Control returns to the caller. The snake game paints the path, or the benchmark script writes one row into the CSV.

That's the entire pipeline. Once you've followed it once end-to-end, reading any individual file in the project becomes much easier.

9. The Snake game, in detail

The Snake game is not just decoration — it is a live laboratory. Every time the snake grows by one segment, the game calls the pathfinder again, because the blocked-cell map has changed. Watching the snake play on the hard board is a visceral way to see how an algorithm behaves when obstacles are constantly appearing and disappearing.

9.1 The game grid

The game uses a fixed 20×20 grid. Each cell has a numeric label:

Value	Meaning
0	EMPTY — walkable
1	BODY — snake body segment (blocks the pathfinder)
2	FOOD — the goal
3	HEAD — the snake's head (the start)
4	WALL — permanent wall from the board layout

9.2 The tick cycle

Every game tick:

1. Copy the 2D board.
2. Temporarily remove the tail cell (it will move forward next, so it is actually free this tick).
3. Build a `PathingGrid` and mark all WALL and BODY cells as blocked.
4. Call `pg.get_path_single_goal(head, food, mode=current_mode)`.
5. Take the first step of the returned path and move the snake head there.
6. If the new head lands on food, grow the snake by one segment and spawn new food.
7. Record timing and expansion counts for the HUD.

9.3 The compare mode

The game has a button that runs *all three* algorithms on the current board state and draws each returned path in its own colour. The helper `run_three_way_compare` does exactly this. When the three paths are equal length (the common case), what differs is not the route but the *effort* it took to compute each route — which is exactly what the HUD reports: per-algorithm running totals of node expansions and wall-clock time.

9.4 The two boards

The file `snake_game_jps4.py` defines two hand-drawn boards:

- `EASY_BOARD_LAYOUT` — a 20×20 board with a thin wall around the outside and only a handful of interior obstacles.
- `HARD_BOARD_LAYOUT` — a 20×20 board with many interior walls forming a maze. This is the board that makes JPS4 look good.

9.5 Why the snake game is a good demo for JPS4

1. The snake's own body is a constantly-changing obstacle. This forces a fresh search every tick, so the per-search cost is what matters.
2. On the hard board, the maze structure creates many long corridors — exactly the pattern JPS4 is tuned for.
3. You can watch the per-algorithm HUD counters grow in real time as the snake plays, which makes the "expansions" metric concrete.

10. The benchmarking methodology

The benchmark scripts exist to answer the question: *across many random grids, which algorithm is fastest and by how much?*

10.1 Fair comparison

The single most important rule in benchmarking: **the same instance must be fed to all three algorithms**. If Dijkstra got an easier random grid than A*, the comparison is meaningless. The code honours this rule by generating a grid once, verifying it is solvable, and then running all three algorithms on that same grid back-to-back before moving on to the next grid.

10.2 What each script does

`benchmark_interim.py`. Single-density run. Default parameters: 50×50 grid, density 0.28 (28% of cells blocked), 40 trials, seed 42. It writes two files into `results/`:

- a CSV with one row per (trial, algorithm),
- a short text summary with mean and standard deviation per algorithm.

`run_preliminary_benchmark.py`. Sweeps density values and records the results for a progress-report-style table. Writes CSV + JSON + a LaTeX table snippet + matplotlib PNGs. This is what produced the *results_combined.png* figure used in the interim progress report.

10.3 Start and goal placement

The helper `pick_start_goal` samples random pairs of free cells and keeps the pair with the largest Manhattan distance. This biases the benchmark toward *long* problems, which is where any pathfinder's differences are visible. Very short problems all finish in microseconds and tell us nothing.

10.4 What is reported

For each trial, three numbers:

- **`ms`** — wall-clock time in milliseconds for the search call.
- **`expansions`** — number of cells popped out of the priority queue. This is the cleanest metric.
- **`path_cells`** — number of cells in the final path. *All three algorithms should produce the same number* on every trial; if they don't, something is wrong with one of the implementations.

10.5 What the interim numbers showed

On 50×50 grids with the JPS4 implementation corrected to match Baum's pruning rules, you should expect roughly:

	15% blocked		**28% blocked**		**40% blocked**	
	Exp.	ms	Exp.	ms	Exp.	ms
Dijkstra	∼2000	∼7	∼1700	∼5	∼900	∼3
A*	∼1000	∼4	∼630	∼2	∼510	∼2
JPS4	∼490	∼4	∼380	∼3	∼320	∼2

(The JPS4 numbers above assume the corrected pruning rules from §5. With the earlier buggy version, JPS4's expansions were artificially inflated at higher densities because the fallback hacks kicked in.)

On wall-clock time, Python's interpreter overhead makes JPS4 look worse than its expansion count alone suggests — its inner loop is more complex. Baum's paper, running on Node.js with a tighter implementation, shows 10–20× speed-ups on real game maps; in Python the ratio is smaller but the expansion-count story is the same.

11. Glossary

Admissible heuristic A heuristic h that never over-estimates the true remaining cost to the goal. Makes A* optimal.

A* A best-first search that uses $f = g + h$ as the priority. Always optimal when h is admissible.

Best-first search A search that always expands the frontier cell with the smallest priority next. Dijkstra, A*, and JPS4 are all best-first searches.

Canonical ordering An artificial rule that forces a unique shortest path among many equivalent ones. JPS4 uses "horizontal first".

Dijkstra's algorithm A best-first search where the priority is just g. No heuristic.

Expansion One pop from the priority queue and the work of generating its successors. The main efficiency metric.

f The priority number on the queue. $f = g + h$ in A* and JPS4; $f = g$ in Dijkstra.

Forced neighbour In JPS4, a neighbour that would normally be pruned but must be kept because an adjacent cell is blocked. A wall has "broken the symmetry".

Frontier (open list) The set of cells waiting to be expanded. Implemented as a priority queue.

g The running cost from the start to the current cell. In 4-connected grids with uniform cost, this is just the step count.

Graph A set of nodes (dots) connected by edges (lines). A grid becomes a graph where free cells are nodes and adjacency gives edges.

h The heuristic — a guess of the remaining cost to the goal. Manhattan distance on 4-connected grids.

Horizontal-first JPS (HJPS/JPS4) Baum's 2025 adaptation of Jump Point Search for 4-connected grids using a horizontal-first canonical ordering.

Jump point In JPS4, a cell where the search must stop and record a new open-list entry: either the goal, or a cell with at least one forced neighbour, or any cell reached via a horizontal move.

Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$. The minimum number of 4-connected steps between two cells in an obstacle-free grid.

Natural neighbour A neighbour that the canonical ordering keeps by default.

Path-symmetry problem The observation that on open 4-connected grids there are many shortest paths and most of them differ only by the order of the moves.

Priority queue A container that always returns the item with the smallest priority. Python's `heapq` module.

Pruning	Skipping the generation of successors that cannot possibly lie on a canonical shortest path.
Union-Find	A data structure that answers "are these two cells in the same connected region?" in near-constant time.
Uniform-cost grid	A grid where every step has the same cost. All the grids in this project are uniform-cost.

End of guide. Return to the code with confidence.
See `pathing_grid.py` *and* `helper.py` *for the implementations described here,*
and `benchmark_interim.py` *to reproduce the numbers in §10.*